Informe: Planificador Curricular

Materia: Lógica Orientada a la Computación y Teoría de Grafos

Carrera: Ingeniería en Sistemas de Información (UTN FRRo)

Integrantes: Matias Xavier Roveri - Manuel SLepoy

Fecha de entrega: 08/11/2026

1. Descripción del problema y decisiones de diseño

El problema abordado consiste en diseñar una herramienta que asista a los estudiantes de Ingeniería en Sistemas de Información a decidir qué materias cursar y en qué orden. La decisión a tomar es la secuencia de cursada de menor riesgo, minimizando la probabilidad de desaprobar o abandonar.

¿Por qué un grafo es una buena forma de modelarlo?

Porque el plan de estudios es como una red de dependencias. Lo modelamos como un Grafo Dirigido Acíclico (DAG), donde:

Nodos: Representan las materias (32 obligatorias + 3 agrupaciones de electivas del Plan 2023 de la UTN FRRo).

Aristas: Representan las correlatividades (de la materia que hay que tener aprobada hacia la que la requiere).

Para tomar la decisión, el sistema no usa solo la distancia o la cantidad de materias, sino que integra un modelo de Machine Learning que estima la probabilidad de aprobar cada nodo (P(aprobar)), y un algoritmo de grafos (Dijkstra) que calcula el camino de menor riesgo acumulado.

2. Descripción del dataset utilizado

Debido a la falta de acceso a datos históricos reales por privacidad institucional, se optó por un dataset sintético.

Generación:

El dataset se genera mediante el script generar_dataset.py. Se simulan 500 estudiantes cursando el plan de estudios. Las variables que influyen en la probabilidad latente de aprobar una materia son:

promedio_previo: Promedio del estudiante hasta el momento.

cant_materias_aprobadas_previas: Nivel de avance en la carrera.

cant_materias_cursando_simultaneo: Carga académica actual.

nota_promedio_correlativas: Rendimiento histórico en las materias previas requeridas. (Si no tiene correlativas, se usa el promedio previo).

es_recursada: Si el alumno ya había cursado y desaprobado la materia.

dificultad: Propiedad fija de la materia (1 a 10) que indica su complejidad intrínseca.

nivel: El año de la carrera al que pertenece la materia (1 a 5).

Regla de generación:

Se calculó un logit en base a estas variables y se pasó por una función sigmoide para obtener la probabilidad real. La fórmula del logit otorga un peso positivo al promedio y a las notas de correlativas, y un peso negativo a la cantidad de materias simultáneas, a la condición de recursada y a la dificultad de la materia. Además, se incluye un intercept base para calibrar el punto de partida.

La fórmula utilizada es:

logit = 0.6 + 0.7*(promedio_previo - 6) + 0.4*(nota_promedio_correlativas - 6) - 0.1*(cant_simult - 4) - 0.8*es_recursada + 0.05*aprobadas_prev - 0.15*(dificultad - 5)

Luego, mediante una distribución de Bernoulli y un margen de ruido para simular deserción, se determinó el resultado final (aprobó, desaprobó, abandonó).

3. Metodología de entrenamiento del modelo

Se optó por un modelo de Regresión Logística (implementado con scikit-learn), justificado por su alta interpretabilidad y porque entrega probabilidades calibradas, algo fundamental para el cálculo de pesos en el grafo. Se comparó previamente con Árboles de Decisión y KNN, pero la regresión logística ofreció un mejor equilibrio.

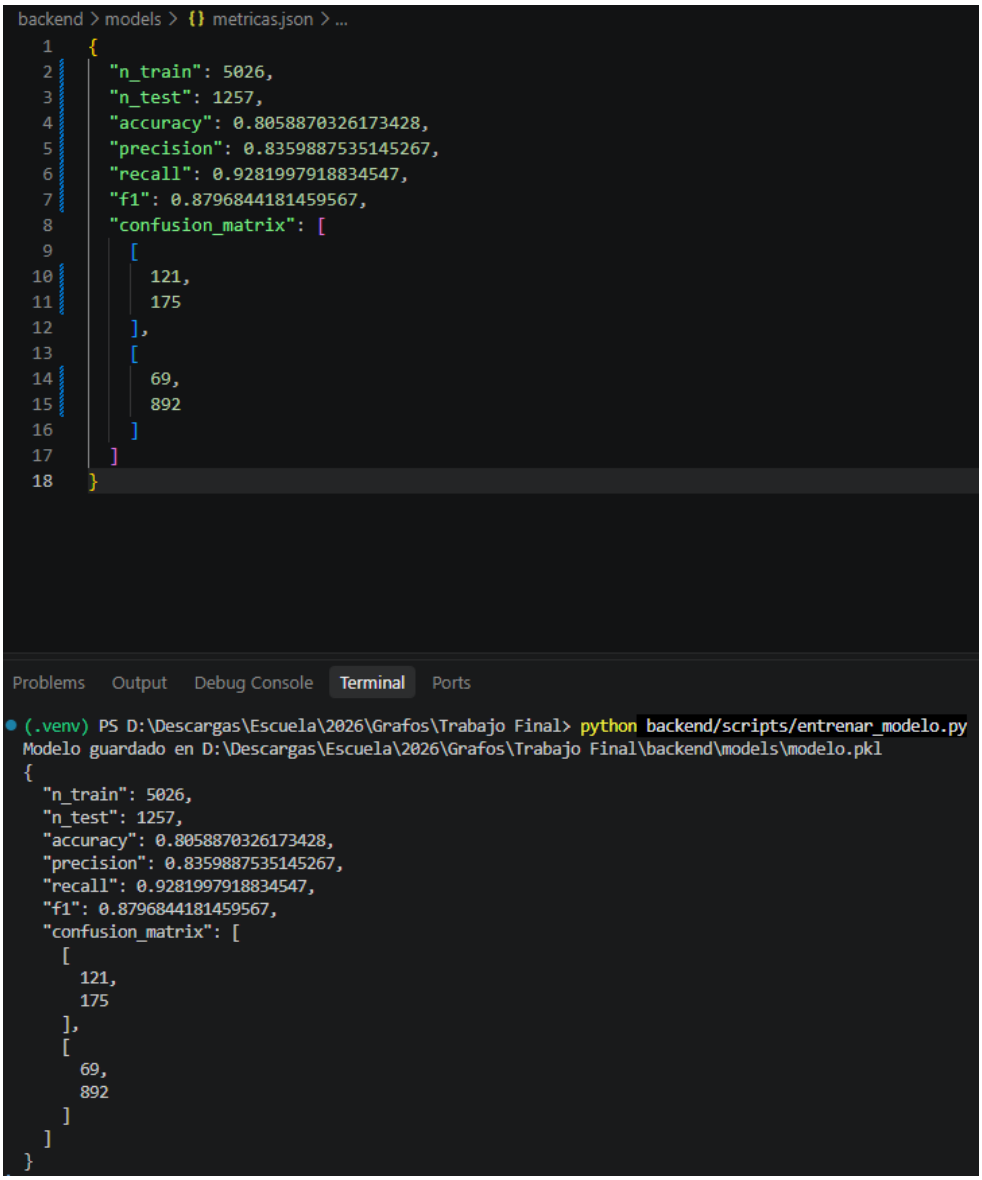
Preprocesamiento: Se utilizó StandardScaler para las variables numéricas (promedio_previo, cant_materias_aprobadas_previas,

cant_materias_cursando_simultaneo, nota_promedio_correlativas, nivel y dificultad) y OneHotEncoder para la variable categórica (es_recursada).

Partición: Se dividió el dataset en 80% para entrenamiento y 20% para test, de forma estratificada para mantener la proporción de aprobados.

Métricas sobre Test:

- Accuracy: 0.806 (80.6%)
- Precision: 0.836
- Recall: 0.928
- F1-Score: 0.880



Análisis de errores: La matriz de confusión muestra un buen desempeño detectando aprobados (pocos falsos positivos), pero todavía tiene margen de mejora prediciendo desaprobados (falsos negativos).

4. Descripción del algoritmo de grafos implementado

Se implementó el algoritmo de Dijkstra a mano (utilizando heapq para la cola de prioridad) para calcular la ruta de menor riesgo desde un nodo ficticio __INICIO__ hacia todas las materias.

Lógica de bloqueo (Correlatividades estrictas):

El sistema recibe el historial del alumno. Solo las materias cuyas correlativas están aprobadas (nota >= 6) tienen una arista desde __INICIO__. Además, si una materia está en la secuencia sugerida pero sus correlativas anteriores en la secuencia aún no han sido aprobadas, el algoritmo la frena y no le permite avanzar a las materias de niveles superiores.

Manejo de aprobadas y recursadas:

El frontend permite cargar notas. Si una correlativa tiene nota menor a 6, se marca como desaprobada. El backend detecta esto, la marca para recursar y bloquea las aristas hacia las materias de las que es correlativa, evitando que el sistema sugiera cursar materias para las que el alumno no está habilitado.

5. Explicación de la integración Modelo - Grafo

El modelo predice la probabilidad PP de aprobar una materia. Para que Dijkstra busque el camino de menor costo, transformamos esa probabilidad en un peso (riesgo) mediante:

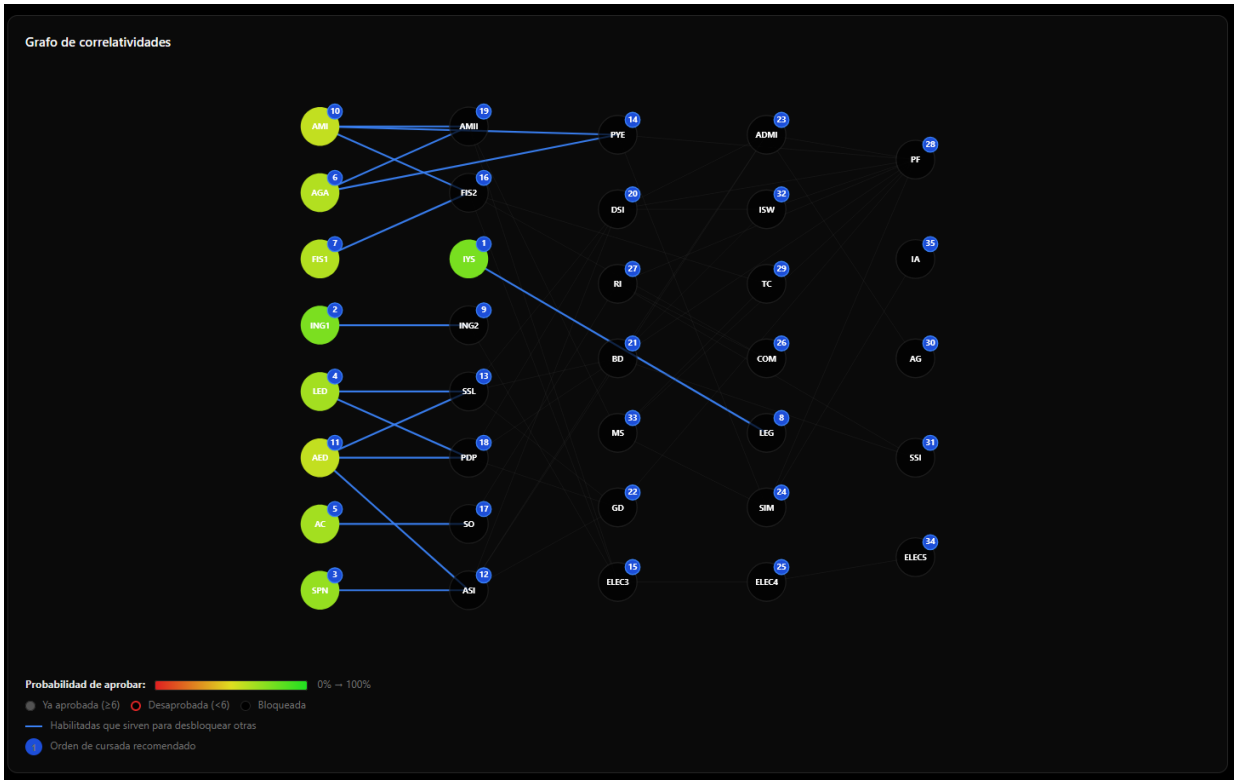
$$\text{Riesgo} = -\ln(P(\text{aprobar}))$$

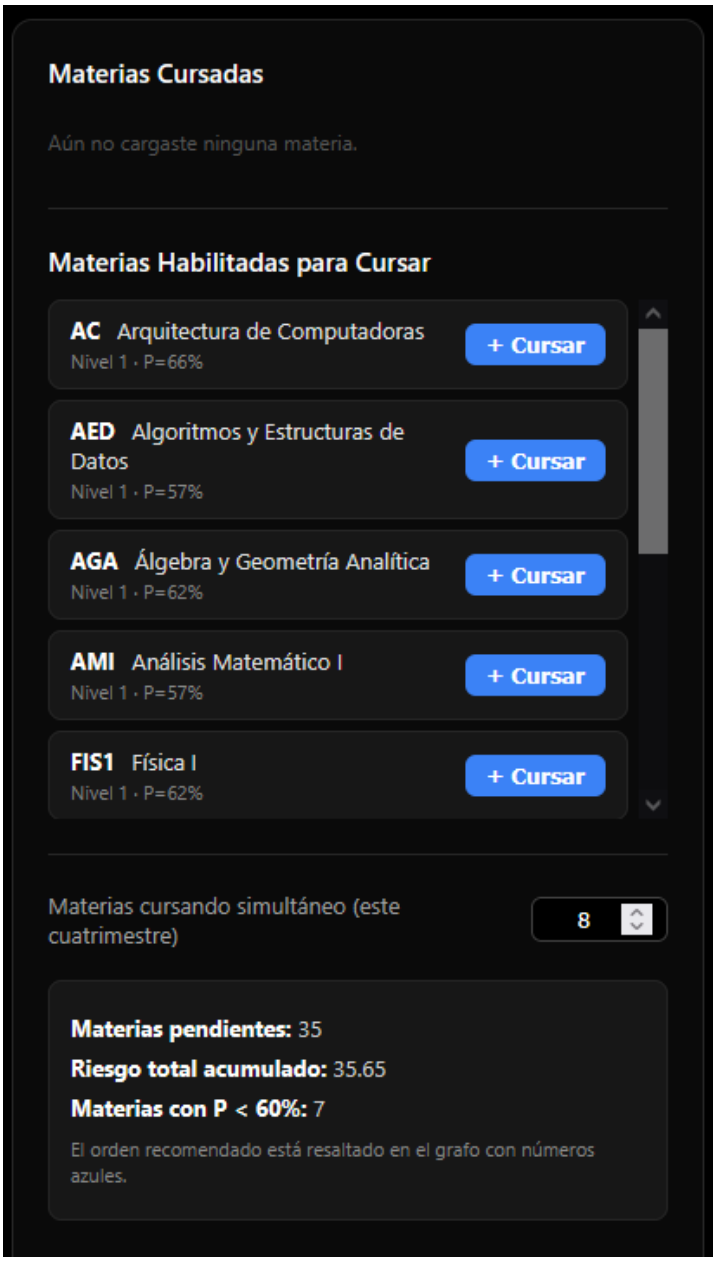
¿Por qué esta transformación es razonable?

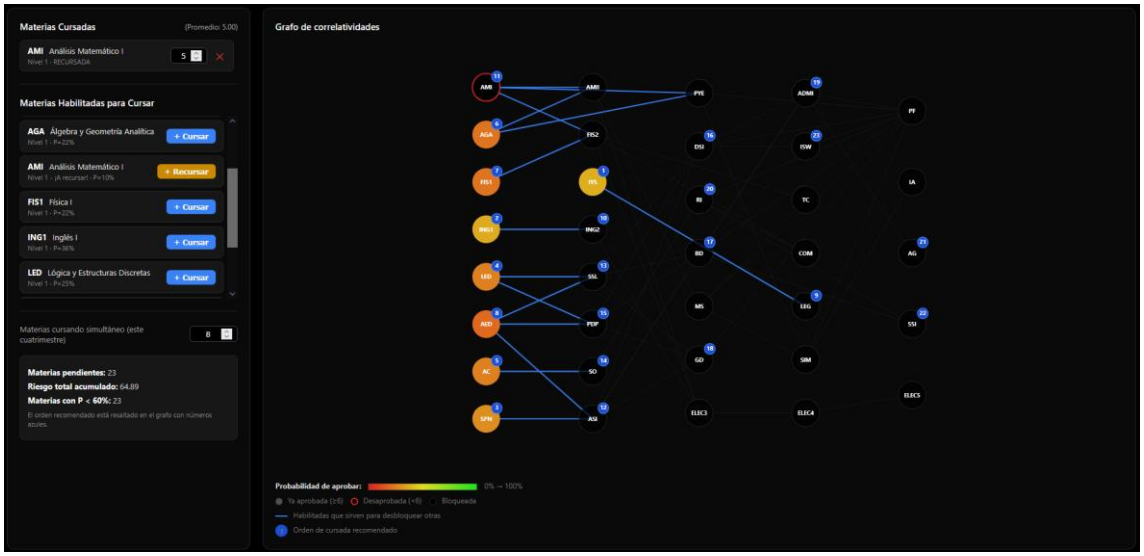
Dijkstra suma los pesos de las aristas. Si usamos $-\ln(P)$, estamos aprovechando la propiedad matemática de que el logaritmo de un producto es la suma de los logaritmos. Al minimizar la suma de los $-\ln(P)$, estamos maximizando el producto de las probabilidades de aprobar todas las materias del camino. Por lo tanto, el algoritmo no elige el camino más corto en cantidad de materias, sino el camino con la mayor probabilidad conjunta de éxito.

Actualización dinámica:

Cada vez que el usuario cambia una nota en la interfaz, el frontend envía el nuevo historial al backend. El backend recalcula el promedio, detecta recursadas, vuelve a predecir las probabilidades de todas las materias, recalcula los pesos y ejecuta Dijkstra, devolviendo la nueva secuencia.




Materias Cursadas
Aún no cargaste ninguna materia.
Materias Habilitadas para Cursar
AC Arquitectura de Computadoras
Nivel 1 · P=66%
+ Cursar
AED Algoritmos y Estructuras de Datos
Nivel 1 · P=57%
+ Cursar
AGA Álgebra y Geometría Analítica
Nivel 1 · P=62%
+ Cursar
AMI Análisis Matemático I
Nivel 1 · P=57%
+ Cursar
FIS1 Física I
Nivel 1 · P=62%
+ Cursar
Materias cursando simultáneo (este cuatrimestre)
8
Materias pendientes: 35
Riesgo total acumulado: 35.65
Materias con P < 60%: 7
El orden recomendado está resaltado en el grafo con números azules.


Materias Cursadas
Materias Habilitadas para Cursar
Grafo de correlatividades

6. Uso de Inteligencia Artificial

Herramienta utilizada: DeepSeek.

¿Para qué se usó la IA?

Para generar el esqueleto inicial del código (backend en Python y frontend en HTML/JS).

Para sugerir la transformación matemática −ln(P) justificando por qué era la mejor forma de transformar probabilidad en peso para Dijkstra.

Para resolver problemas de integración, como la configuración de CORS en FastAPI y el diseño visual del grafo (coordenadas SVG y gradientes de colores HSL).

Prompts útiles: "¿Cómo transformo una probabilidad en un peso para Dijkstra de forma que minimizar la suma equivalga a maximizar la probabilidad conjunta?" y "¿Cómo dibujo un grafo en SVG usando Vanilla JS y calculo las posiciones de los nodos por nivel?".

¿Para qué NO se usó la IA?

La IA no se usó para corregir el presente informe ni para corregir el archivo README.md (esos fueron redactados y revisados enteramente por nosotros).

La IA no definió las reglas del dataset sintético ni las correlatividades del plan de estudios (fueron extraídas e interpretadas por nosotros).

La IA no implementó la lógica de validación estricta de correlatividades (es decir, verificar que todas las correlativas estén aprobadas y no solo una).

Errores de la IA: Al principio, la IA sugirió usar la librería networkx para implementar Dijkstra. Nosotros nos dimos cuenta de que la consigna exigía implementar el algoritmo a mano, así que lo descartamos y lo programamos con heapq. Además, un primer código de la IA fallaba al validar las correlatividades en el grafo, porque si una materia tenía 3 correlativas, el algoritmo las ignoraba si encontraba un camino por al menos una de ellas. Lo detectamos al probar el sistema y corregimos la lógica para que exija el cumplimiento del 100% de las correlativas.

Reflexión: La IA fue una excelente herramienta para acelerar la escritura del código y proponer soluciones visuales. Sin embargo, no reemplazó el entendimiento del problema; de hecho, sus errores nos obligaron a comprender a fondo la lógica del grafo para corregirla.

7. Limitaciones conocidas y posibles extensiones

Dataset: Al ser sintético, carece de las complejidades y sesgos del mundo real. Como extensión, se podría adaptar el sistema para consumir datos reales anonimizados del SIU-Guaraní de la facultad.

Simplificación de correlatividades: El reglamento real distingue entre "regularizada" y "aprobada". El sistema actual simplifica esto a un solo estado (nota >= 6). Una mejora sería agregar el estado de "cursando" para que el alumno pueda simular su situación a mitad de cuatrimestre.

Modelado de materias: El modelo no distingue entre dos materias del mismo nivel con distinta dificultad real (porque no recibe el materia_id). Una extensión sería incorporar estadísticas históricas de aprobación por materia específica como feature.

Interactividad: Se podría extender el frontend para permitir arrastrar y soltar los nodos (drag & drop), o simular múltiples cuatrimestres a la vez, mostrando una planificación a 2 o 3 años vista.

Agrupación de materias electivas: En el Plan 2023, la oferta de materias electivas varía año a año según los cupos del Departamento de Sistemas. Para que el grafo no se rompa ni sea inconsistente entre distintos años, se simplificó el bloque de electivas agrupándolas en tres nodos genéricos según su nivel (ELEC3, ELEC4 y ELEC5), con correlatividades encadenadas entre sí. Una extensión futura muy interesante del sistema sería agregar cada electiva real (Backend, DevOps, Arquitectura de Software, etc.) como un nodo independiente en el grafo, permitiendo que el estudiante elija la que más le convenga según su perfil.